

# Working Memory with Polychronous Chains

Laurent Ursule Perrinet

`laurent.perrinet@univ-amu.fr`

Institut de Neurosciences de la Timone (UMR 7289)

Aix Marseille Univ, CNRS

27 boulevard Jean Moulin, 13005 Marseille, France

<https://laurentperrinet.github.io/perrinet-26>

September 1, 2026

Working memory — the ability to store and recall precise temporal patterns of neural activity — remains an open challenge for spiking neural networks (SNNs). We propose a recurrent SNN of  $N = 1024$  neurons in which each synapse is equipped with  $D = 41$  heterogeneous delays, parameterised as a weight tensor  $\mathbf{W} \in \mathbb{R}^{N \times N \times D}$  and trained end-to-end with surrogate-gradient backpropagation through time. Each stored pattern is represented as a sequential chain of overlapping Spiking Motifs: contiguous context windows of length  $D$  that uniquely predict the activity at the next time step. A closed-form Hebbian initialisation, derived by deconvolving the LIF membrane response and targeting a sub-threshold membrane potential value superior to the threshold  $\vartheta$ , achieves an accuracy as measured by the  $F_1$ -score relative close to that expected when noise is present and before any gradient step on a benchmark of  $M = 16$  patterns of duration  $T = 1000$  ms. With learning, the network tolerates up to 25% bit-flip noise, and reaches  $F_1$  scores closer to the value expected from the noise level. These results demonstrate attractor-like retrieval dynamics consistent with hippocampal pattern completion, low-dimensional collective dynamics of population activity on manifold or mesoscopic traveling waves in sensory areas. These results show that heterogeneous synaptic delays are an efficient and scalable substrate for working memory in SNNs, with direct implications for neuromorphic edge deployment.

**Keywords:** Spiking Neural Networks, Polychronization; Heterogeneous delays, Working memory.

## 1 Introduction

A century ago, [?] established that neural communication relies on all-or-none action potentials, *spikes*. Whether information is encoded in precise spike timing or in the coarser measure of instantaneous firing rate remains debated [?] — a question that becomes especially sharp for working memory, where the brain must maintain and manipulate information across delays of seconds. In traditional artificial neural networks, long-range temporal dependencies are addressed by gated recurrent unit (GRU) architectures or Transformers [?], which decouple temporal binding range from the timescale of individual units. The analogous problem for spiking neural networks (SNNs) — binding activity patterns separated by intervals far exceeding

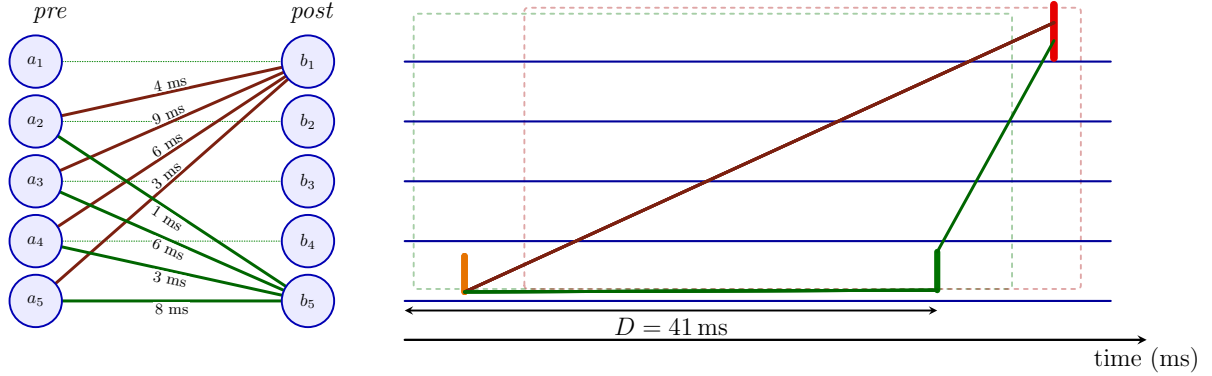


Figure 1: **Working memory as a chained sequential spike prediction via heterogeneous delays.** *Left:* A recurrent HD-SNN of five neurons numbered 1–5, connected by two subsets of recurrent connections that define two Spiking Motifs from the pre-synaptic side (neurons  $a$ ) to the post-synaptic side (neurons  $b$ ): dark-red connections from  $a_2$ ,  $a_3$ ,  $a_4$ , and  $a_5$  target  $b_1$  with delays 4, 9, 6, 3 ms; green connections from  $a_1$ ,  $a_2$ ,  $a_3$ ,  $a_4$  target  $b_5$  with delays 1, 6, 3, 8 ms. *Right:* Raster plot illustrating how the network predicts successive spikes from a past context window. *Step 1 — first prediction (green dashed box):* the orange input spikes emitted at staggered times by  $a_2$ – $a_5$  within the first context window (dashed green box) constitute a heterogeneous Spiking Motif. The green delays are tuned so that all four spikes converge *synchronously* at  $b_5$ , crossing the firing threshold and producing the green output spike at a precise time. *Step 2 — second prediction (dark-red dashed box):* the green spike of  $a_5$  re-enters the network as a new input and, together with the spikes now visible in the shifted context window (dark-red dashed box), is routed through the dark-red delays in order to converge synchronously on  $b_1$ , generating a new output spike. Because output spikes re-enter the same population, detecting one Spiking Motif provides the context for predicting the next spike in the sequence, implementing working memory as a chain of temporal predictions grounded in the preceding context window.

a single neuron’s time constant — lacks a general solution and remains a key challenge for biologically plausible sequence processing [? ].

[?] extended the synfire framework by introducing heterogeneous axonal delays in a recurrent population of spiking neurons trained with spike-timing-dependent plasticity (STDP). The resulting *polychronous neuronal groups* (PNGs) — time-locked, non-synchronous firing patterns at millisecond precision — vastly outnumber the neurons in the network, yielding a large short-term memory capacity exploited for visual feature binding [?] and sequence classification [? ]. [?] showed that short-term plasticity can cue a PNG’s reactivation and sustain a working memory. However, storing spike patterns of arbitrary length is inherently difficult: the sequential structure of the task means each context window is not an independent query but is itself the output of the network at the previous step, so small errors at early time steps propagate and amplify towards the end of the sequence — a compounding difficulty absent from static associative memory like Hopfield networks.

The limitation of the original model from [?] is that delays are fixed rather than learned. Recent work addresses this: [?] demonstrated fast, energy-efficient inference on neuromorphic hardware using first-spike-time coding, while [?] introduced gradient-based delay learning via dilated convolutions with learnable spacings (DCLS), achieving sparse solutions in which only  $\sim 20\%$  of delay channels remain active. Applied to event-based vision, heterogeneous learned delays in a single spiking layer suffice for fast motion detection [? ], scalable to always-on object recognition [? ]. Building on this, *Spiking Motifs* (SMs) generalise PNGs to a gradient-trainable, hardware-compatible framework by replacing fixed STDP rules with end-to-end learned delays, here referred to as Heterogeneous-Delay SNNs (HD-SNNs) [? ]. However these motifs have a short time span equal to the range of possible delays, while it has been shown in mice stimulations that sequences of the order of seconds influence neural activity [? ]. [?] recently extended this to motifs of arbitrary length by chaining sub-motif detectors with bounded synaptic delays, achieving robust encoding of complete Spiking Heidelberg Digits patterns.

The present work extends the chain-of-motifs framework of [?] to a fully recurrent topology for working memory. Rather than relying on fixed delays and unsupervised STDP as in the original polychronisation model, the network is trained end-to-end with surrogate gradients [? ]: once primed by a short trigger window of  $D$  steps, it autonomously recalls arbitrary-length spike sequences, bridging polychronisation theory with gradient-based machine learning.

## 2 Methods

### 2.1 Recurrent HD-SNN architecture

In a previous work, we have demonstrated that a feedforward HD-SNN can learn to detect Spiking Motifs [? ]. However, this architecture is limited to motifs of length  $D$ , the maximum synaptic delay. To store and recall longer sequences, it is possible to extend this architecture to a fully recurrent topology, where each neuron receives inputs from all other neurons across all delays. Our core concept is to retrieve a memory progressively, in which each motif of length  $D$  would allow to predict the next spikes in the memory to retrieve. This allows the network to generate a chain of overlapping Spiking Motifs, where each motif provides the context for predicting the next spikes in the sequence (see Figure ??).

In practice, we will study a recurrent Heterogeneous-Delay SNN (HD-SNN) which consists of a population  $\mathcal{A}$  of  $N = 1024$  leaky integrate-and-fire (LIF) neurons implemented in `snnTorch` [?] and simulated over  $T = 1000$  steps of 1 ms each (total duration: 1 s). Extending the feedforward Spiking Motif Detector [? ], our architecture will include full recurrence: each neuron  $j \in \mathcal{A}$  integrates inputs from *all* other neurons across *all* preceding  $D = 41$  time steps. Such network is the most general for learning purposes, and at inference, pruning of the synapses may be added to gain from the sparseness of synapses on the energy budget. Moreover, the choice for

the maximum delay value  $D$  is justified from biological observations and similar to the value used in the original polychronisation study [? ]. The membrane potential  $u_j(t)$  evolves as

$$u_j(t) = \beta \cdot u_j(t-1) \cdot (1 - s_j(t-1)) + \sum_{i=1}^N \left( \sum_{d=1}^D \mathbf{W}_{j,i,d} \cdot s_i(t-d) \right), \quad (1)$$

where  $\beta \in (0, 1)$  is the membrane decay constant ( $\beta = 0.8$  in our simulations, that is, a time constant of  $\tau = 1/\ln(\frac{1}{\beta}) \approx 4.5$  steps),  $\mathbf{W}_{j,i,d}$  is the synaptic weight to neuron  $j$  from neuron  $i$  at delay  $d$ , and  $s_j(t) \in \{0, 1\}$  is the spike indicator  $s_j(t) = \mathbb{1}[u_j(t) \geq \vartheta]$ , that is,  $s_j(t) = 0$  if  $u_j(t) < \vartheta$  and a spike is emitted  $s_j(t) = 1$  when  $u_j(t)$  exceeds the threshold  $\vartheta$ . After spiking, the membrane potential that is accumulated in Equation ?? is reset to zero.

In our implementation, the spike history of all  $N$  neurons over the past  $D$  steps is concatenated into a flattened context vector of dimension  $N \cdot D$  at each time step, so that  $\mathbf{W}$  is reshaped as  $\mathbf{W} \in \mathbb{R}^{N \times (N \cdot D)}$ . This is equivalent to the temporal convolution that we used previously in a feedforward setting [? ], here extended to a fully recurrent topology. The HD-SNN thus maps onto a standard static computational graph that can be trained with surrogate gradient descent as classical SNNs (see Figure ??, left panel).

## 2.2 Synthetic dataset

To assess working-memory capacity in all generality, we generated a dataset of  $M = 16$  distinct target patterns  $\mathbf{S}^{(\mu)} \in \{0, 1\}^{N \times T}$ . Following previous work [? ], each pattern is generated using a stochastic spiking pattern generator with both a structured and background component. Specifically, for each pattern we define a *frozen* pattern as  $P^{(\mu)} \in \{0, 1\}^{N \times T}$  with  $P^{(\mu)} \sim \text{Bernoulli}(p_A)$  where we set  $p_A = 1.6 \times 10^{-4}$  spike per neuron per time step (0.16 Hz if we consider 1 ms time bins) as the average spike probability. This follows the estimation of the average spiking frequency in the human brain [? ].

To account for the stochasticity inherent to neural activity, we further include a balanced bit flipping mechanism with probability  $p_{\text{flip}} = 0.05$ . This is defined by the binary mask  $M^{(\mu)} \in \{0, 1\}^{N \times T}$  of bit flips locations :  $M^{(\mu)} \sim \text{Bernoulli}(p_{\text{flip}})$ . Values are left untouched where  $M^{(\mu)} = 0$  and will be drawn with probability  $p_A$  where  $M^{(\mu)} = 1$ . Finally, the spike patterns  $S^{(\mu)} \in \{0, 1\}^{N \times T}$  are those of the frozen pattern  $P^{(\mu)}$  except where  $M^{(\mu)} = 1$  for which they are obtained by drawing with probability  $p_A$ . Formally, this can be expressed as:

$$\mathbf{S}^{(\mu)} \sim P^{(\mu)} \cdot (1 - M^{(\mu)}) + \text{Bernoulli}(p_A) \cdot M^{(\mu)} \quad (2)$$

Note that the bit flipping mechanism introduces a stochasticity into the structure of the spike patterns, however, it does not change the average firing rate. Indeed, since the generation of  $\mathbf{P}^{(\mu)}$  and  $\mathbf{M}^{(\mu)}$  are independent, the expected firing rate  $E(\mathbf{S}^{(\mu)})$  is equal to  $p_A \cdot (1 - p_{\text{flip}}) + p_A \cdot p_{\text{flip}} = p_A$ .

## 2.3 Working memory task: memory cuing

Given a set of target patterns as defined above, a generic working memory task consists of training the network to reproduce the target pattern that follow a short *trigger* window. Formally, given  $\mathbf{S}^{(\mu)}$ , each contiguous window of length  $D$ , i.e. the slice  $\mathbf{S}^{(\mu)}[:, t-D : t-1]$ , constitutes a unique context that may be used to predict the spiking activity  $s_j(t)$  of all neurons  $j$  at time  $t$  (see Figure ??). The network is therefore trained to generate a sequential chain of overlapping Spiking Motifs [? ? ] that faithfully reproduces the target. Memory retrieval is cued by clamping all neuron activities to the target pattern during the trigger window which corresponds to the initial  $D$  time steps ( $0 \leq t < D$ ), that is, corresponding to one elementary Spiking Motif (see Figure ??, right panel); the goal of the network is then to generate the remainder of the sequence autonomously.

As an accuracy metric, we will use the mean  $F_1$  score between the predicted and target spike trains, averaged over time (in the prediction window), neurons and patterns. The  $F_1$  score is the harmonic mean of precision (proportion of predicted activity that are correct) and recall (proportion of predicted activity that is successfully reproduced). Knowing a priori that average neural activity is sparse, maximising  $\mathcal{L}$  simultaneously penalises over-active networks (low precision) and silent networks (low recall).

Applied to the synthetic patterns, knowing  $p_{\text{flip}}$  then one may first compute the precision as the ratio of expected true positives to the sum of true positives and false positives, that is,  $\frac{p_A \cdot (1 - p_{\text{flip}} + p_A \cdot p_{\text{flip}})}{p_A \cdot (1 - p_{\text{flip}} + p_A \cdot p_{\text{flip}}) + p_{\text{flip}} \cdot (1 - p_A) \cdot p_A} = 1 - p_{\text{flip}} \cdot (1 - p_A)$ . Then, recall is equal to precision as the average ratio of false negatives  $p_A \cdot p_{\text{flip}} + p_{\text{flip}} \cdot (1 - p_A) \cdot p_A$  is equal to that of false positives (remember that the average frequency is not changed by the balanced bit flip operation). As a result, the  $F_1$  score between the frozen and perturbed patterns equals the precision and recall and is given by:

$$F_1 = 1 - p_{\text{flip}} \cdot (1 - p_A) \quad (3)$$

This will provide a theoretical bound for the average  $F_1$  score that can be achieved by the network, as it is of course impossible to recover exactly the original frozen pattern (when  $p_{\text{flip}} = 0$ ) from a perturbed one (when  $p_{\text{flip}} > 0$ ).

## 2.4 Analytical weight initialisation

From Equation ??, by normalizing the potential to arbitrary units, we wish the membrane potential to reach a sub-threshold value close to one whenever a spiking motif from a target pattern occurs. Before training, the weight tensor  $\mathbf{W}$  can be initialised analytically by treating this task as a linear regression problem. For each target pattern  $\mathbf{S}^{(\mu)} \in \{0, 1\}^{N \times T}$ , the context window at time  $t$  is the flattened vector  $\mathbf{c}^{(\mu)}(t) \in \{0, 1\}^{N \cdot D}$  containing the spike history of all neurons over steps  $t - D$  to  $t - 1$  and the corresponding desired output  $\mathbf{I}(t) \in \{0, 1\}^N$ . Stacking all  $K = M \cdot (T - D)$  context-output pairs across patterns and time steps yields

$$\mathbf{C} \in \{0, 1\}^{(N \cdot D) \times K}, \quad \mathbf{I}^* \in \mathbb{R}^{N \times K},$$

and the initialisation problem becomes  $\mathbf{W} \mathbf{C} \approx \mathbf{I}$ .

The choice of regression output  $\mathbf{I}$  requires care. A naive choice is the binary spike matrix  $\mathbf{S}$  itself, which would amount to ask the network to produce a current equal to the target spike at each step. However, the synaptic current  $I_j(t) = \sum_{i,d} \mathbf{W}_{j,i,d} \cdot s_i(t - d)$  is not compared directly to  $s(t)$ : it is first *filtered* by the membrane, which acts as a causal first-order IIR lowpass with transfer function  $H(z) = 1/(1 - \beta \cdot z^{-1})$  and impulse response  $h(t) = \beta^t$ . The correct output is therefore obtained by deconvolution. The inverse filter of  $H(z)$  is  $H^{-1}(z) = 1 - \beta \cdot z^{-1}$ . Applied to a target trajectory that reaches 1 at  $t$  and resets immediately, the optimal input current is therefore

$$I_j(t) = s_j(t) - \beta \cdot s_j(t - 1). \quad (4)$$

This is a *biphasic* waveform: 1 at each target spike time  $t$  and  $1 - \beta$  exactly one step later, cancelling the residual membrane charge left by the leak. For  $\beta \rightarrow 0$  the biphasic correction collapses to a scaled impulse; for  $\beta = 0.8$  the negative lobe is substantial and significantly reduces spurious spikes.

Since the system is strongly underdetermined ( $K \ll ND$ ), the minimum- $\ell_2$ -norm solution is the Moore–Penrose pseudo-inverse  $\mathbf{W}^* = \mathbf{I}^* \cdot \mathbf{C}^+$ . With sparse neural activity at rate  $p_A = .16 \cdot 10^{-3}$ , consecutive context windows are nearly orthogonal, so the Gram matrix satisfies  $\mathbf{C} \mathbf{C}^\top \approx N \cdot D \cdot p_A \cdot \mathbf{I}$ , giving the closed-form initialisation

$$w_{i,j,d} = \frac{1}{N \cdot D \cdot p_A \cdot M} \sum_{\mu=1}^M \sum_{t=D+1}^T s_i^{*(\mu)}(t - d) \cdot (s_j^{*(\mu)}(t) - \beta \cdot s_j^{*(\mu)}(t - 1)). \quad (5)$$

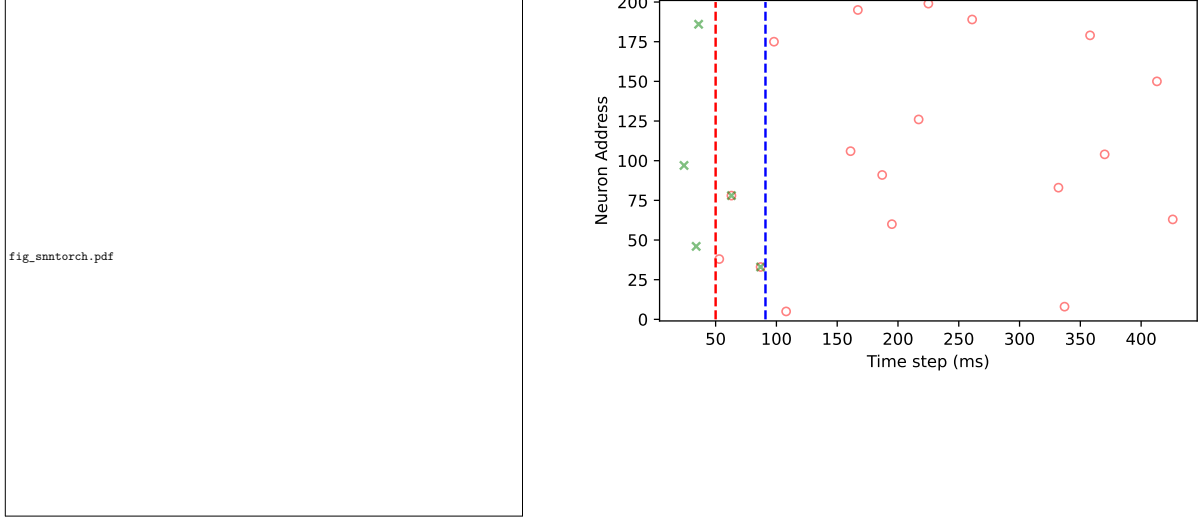


Figure 2: Network architecture and a sample target pattern. *Left:* Unrolled view of the recurrent HD-SNN over time (one column representing one time step), illustrating delayed connections between neurons: Blue arrows indicate synaptic connections that integrate past activity across heterogeneous delays; for clarity only one delay is shown. *Right:* One representative target spike pattern ( $N = 1024$  neurons,  $T = 1000$  time steps, of which we show 256 neurons and 400 time steps); each red circle marks one target spike. The goal of the network is to memorise  $M = 16$  such patterns. Each trial is flanked by  $N_{\text{pretime}} = 50$  time steps of spontaneous background activity (random spikes at rate  $p_A$ ) before and after the stimulus. The trigger window (green crosses between the dashed lines) consists of the first  $D = 41$  time steps of the target pattern clamped to the network input; the recurrent network then autonomously predicts the remaining spikes.

Each weight  $w_{i,j,d}$  is proportional to the cross-correlation between the deconvolved spike train of neuron  $j$  (scaled by  $\vartheta_0 = 0.8$ ) and the raw spike train of neuron  $i$  at delay  $d$ , averaged over all  $M$  patterns.

We will set the threshold at  $\vartheta = 0.8$  at each target spike time  $t$ , reserving a safety margin  $\delta = 1 - \vartheta = 0.2$ . This margin prevents spurious firings from partial contexts and, crucially, places the membrane in the regime where the surrogate gradient is near its maximum, making the first gradient step maximally informative (see Appendix ?? for the full derivation and its relationship to capacity) The  $\vartheta_0$  factor ensures that a perfect context produces  $u_j(t^*) = \vartheta_0$ , leaving a margin  $\delta = 0.2$  below the threshold. This mirrors spike-timing-dependent plasticity (STDP) [?] and classical Hebbian learning [?], with the additional  $-\beta$  correction accounting for the membrane leak (full analysis in Appendix ??). At inference time the network receives its own generated context rather than the ground truth, so errors can accumulate over time — the primary source of difficulty analysed in Section ??.

## 2.5 Training

The loss is defined as  $\mathcal{L} = 1 - F_1$ , such that minimizing this loss (with a maximum of 1 and minimum of 0) maximises the mean  $F_1$  score across all  $M$  target patterns. Learning is carried out with surrogate-gradient BPTT [?], using a fast-sigmoid surrogate with sharpness parameter  $\alpha = 12$ . Weights are optimised with SGD, using a cosine learning-rate schedule with warmup, as used for instance in [?], from a base learning rate  $\eta_{\text{base}} = 20.0 \times 10^{-3}$  to a final learning rate  $\eta_{\text{final}} = 400 \times 10^{-6}$ , with 16 warmup epochs over 256 total epochs. Dropout regularization with probability 0.10 is applied. The validation loss is monitored for detecting overfitting.

The gradient-based learning in our recurrent HD-SNN shares fundamental principles with biological STDP, despite operating at different scales. Both mechanisms exploit temporal correlations in spike timing to adjust synaptic strengths. However, while STDP implements local pairwise updates based on individual spike timings, our approach performs global optimisation that effectively learns the collective interaction of multiple delayed spike trains across the entire network. The analytical initialisation (see Equation ??) reveals this connection most clearly: it implements a Hebbian-like rule that is mathematically equivalent to averaging STDP updates across all stored patterns, suggesting that gradient descent in this context can be viewed as a principled, globally optimal implementation of temporally structured synaptic plasticity.

Our implementation is available at <https://github.com/laurentperrinet/MNESIS>. The recurrent HD-SNN was implemented using the `snnTorch` [?] library with custom extensions for heterogeneous delay handling. The implementation leverages PyTorch with the Metal Performance Shader (MPS) backend for accelerated computation on Apple Silicon (M3 Ultra chip featuring 32 CPU cores and 80 GPU cores, with 512 GB of unified memory architecture) and further tested with the CUDA backend on the Jean Zay supercluster at GENCI. The MPS and CUDA backends enabled efficient training and simulation of the large recurrent networks required for working memory tasks, leveraging the parallel processing capabilities of the GPU cores while maintaining low-latency access to the substantial memory resources necessary for storing all tensors’ parameters. Training typically converged within 256 gradient steps, requiring approximately 5 minutes of computation time per training.

## 3 Results

### 3.1 Training dynamics and results

Training the recurrent HD-SNN with surrogate-gradient BPTT [?] consistently maximised the mean  $F_1$  score across all  $M = 16$  target patterns to approximately 0.9, with  $N = 1024$  neurons,  $D = 41$  delays,  $T = 1000$  time steps,  $p_A = 1.6 \times 10^{-4}$ , and  $\beta = 0.8$  (time constant  $\tau \approx 4.5$  ms). Without the analytical initialisation, two characteristic failure modes arise during early training: episodes of complete silence, in which the network emits no spikes, and over-activation, in which nearly all neurons fire at every time step. Both are well-known pathological attractors of recurrent SNNs [?]. The  $F_1$  score is the harmonic mean of precision and recall, and maximising it therefore penalises both failure modes symmetrically: low precision corresponds to an over-active network and low recall to a silent one. The cosine learning-rate schedule [?] helped guide the network away from these extremes towards a balanced, sparse-firing regime.

Remarkably, the analytical weight initialisation alone (Eq. ??) already achieves high  $F_1$  scores before any gradient step, demonstrating that the closed-form Hebbian solution provides a strong baseline for recall on this benchmark. The Gram-matrix approximation  $\mathbf{C}\mathbf{C}^\top \approx N \cdot D \cdot p_A \cdot \mathbf{I}$  reduces the pseudo-inverse to a simple cross-correlation, yielding a  $\approx 5\times$  speed-up with identical accuracy; this approximation is used throughout. Gradient training serves primarily to improve robustness: errors arise when the trigger window is corrupted (Section ??), and learning helps mitigate their accumulation.

### 3.2 Effect of hyperparameters

We performed a systematic sweep over the key hyperparameters and report their effect on the mean  $F_1$  score at convergence.

**Surrogate sharpness  $\alpha$ .** The sharpness of the surrogate gradient had the largest single effect on performance. Low values of  $\alpha$  produced gradients that were too smooth to discriminate precise spike times, while excessively high values caused gradient vanishing. The default value used in this study is  $\alpha = 12$ , consistent with the robust training regime observed in our parameter scans.

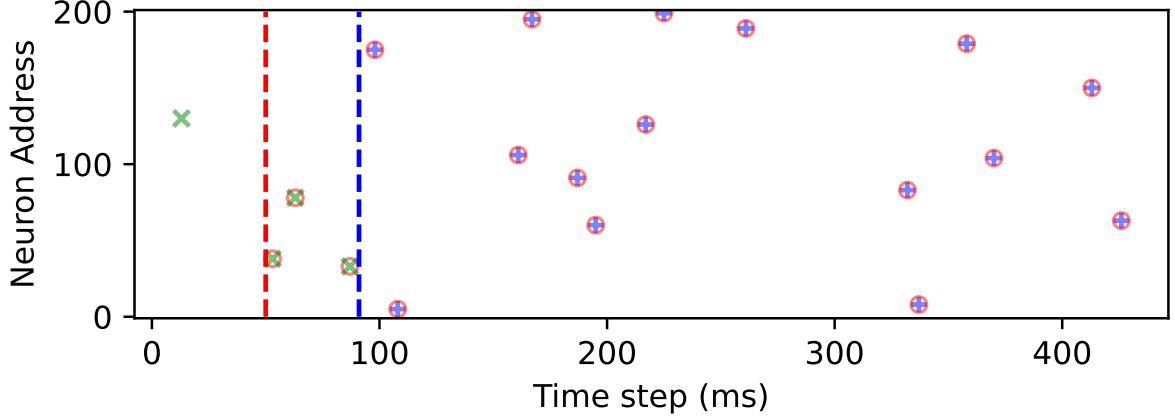


Figure 3: Pattern retrieval from the recurrent HD-SNN. After clamping the network with input spikes (green crosses) with 50 steps of spontaneous activity (left of the red dashed line) and a trigger window consisting of the initial sequence of one of the  $M = 16$  target patterns ( $0 \leq t < D = 41$ , delimited by the dashed red and blue line), we simulate the network after training output the duration the full sequence duration ( $T = 1000$  steps). We observe that the raster plot shows network activity (blue crosses,  $N = 1024$  neurons) which perfectly matches the target pattern which was cued during the trigger window. Note that the target spikes after the trigger window (red circles right of the blue dashed line) are hidden from the network.

**Target firing rate.** The best recall was obtained at low target firing rates (default 0.16 Hz per neuron), confirming that sparse activity facilitates stable recurrent dynamics [? ?]. Higher firing rates led to increased inter-pattern interference and degraded precision, echoing the capacity bounds reported for polychronous neuronal groups [? ].

### 3.3 Retrieval of multiple spike patterns

Training was not uniform in time: the network first reproduced spikes immediately following the trigger window ( $0 \leq t < D$ ), then progressively extended correct recall towards later time steps, as shown in Figure ???. This behaviour is a direct consequence of the sequential structure of the task: because each context window  $[t - D, t - 1]$  uniquely predicts the activity at time  $t$ , errors at early time steps propagate into all subsequent predictions, making temporal proximity to the trigger window the primary driver of learning progress.

To evaluate the network’s ability to retrieve and distinguish multiple stored patterns in sequence, we concatenated all  $M = 16$  patterns into a single long raster, each trial flanked by  $N_{\text{pretime}} = 50$  steps of spontaneous background activity at rate  $p_A$  before and after the stimulus (Figure ??). Each pattern was triggered by its individual trigger window, and retrieval was evaluated by a sliding-window  $F_1$  score over time. The  $F_1$  score rose sharply at the onset of each trigger window, remained high throughout each pattern’s duration, and dropped back to the background level during the inter-trial intervals, confirming that the network correctly associates each trigger window with its corresponding stored pattern without cross-interference between the  $M = 16$  stored memories.

### 3.4 Working memory and robustness to perturbations

A hallmark of biological working memory is graceful degradation: recall should remain possible under incomplete or corrupted retrieval cues, mirroring the tolerance of hippocampal pattern



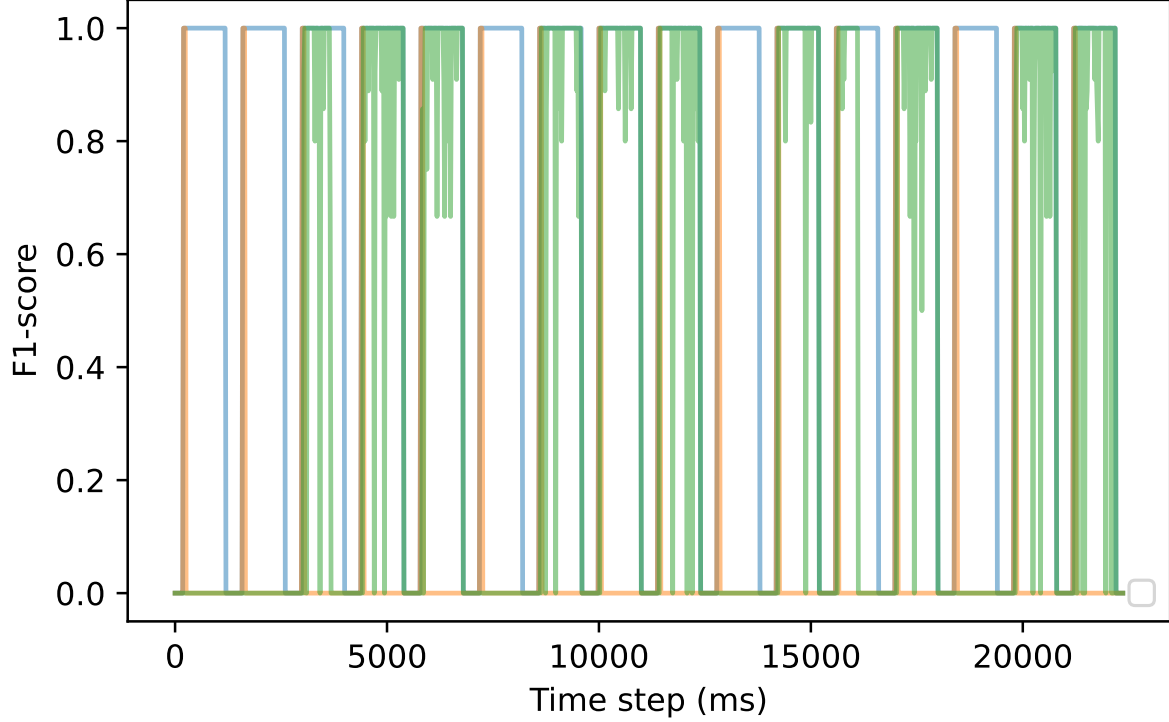


Figure 4: Sequential retrieval of the  $M = 16$  stored patterns. The sliding-window  $F_1$  score (blue) is shown as a function of time, together with the onset indicator of each trigger window (green dashes) and the corresponding pattern mask (red dashes). The  $F_1$  score rises sharply at each trigger-window onset and remains high for the full pattern duration ( $T = 1000$  steps  $\sim 1$  second), then returns to baseline during the inter-trial intervals, demonstrating selective retrieval of all stored memories in sequence without cross-interference.

completion to partial input [? ]. We therefore evaluated three distinct perturbations of the trigger window, each probing a different dimension of robustness (Figure ??).

**Robustness to spike noise (Figure ??, left).** We corrupted the trigger window by independently flipping each spike with probability  $p_{\text{flip}} \in [0, 1]$  (bit-flip noise). At  $p_{\text{flip}} = 0.25$ , the network still achieved an  $F_1$  score of  $\approx 0.627$  (precision  $\approx 0.898$ , recall  $\approx 0.481$ ), demonstrating a reasonable tolerance to noise in the trigger window. The  $F_1$  score decreases monotonically as  $p_{\text{flip}}$  increases, reaching chance level only near  $p_{\text{flip}} = 0.5$  where the trigger window is entirely randomised. This behaviour is consistent with the network operating as an attractor: small perturbations of the initial state are corrected by the recurrent dynamics, which converge back to the stored pattern.

**Robustness to trigger-window duration (Figure ??, middle).** We varied the duration of the trigger window from 0 to  $D - 1 = 40$  steps, scanning from no cue at all to a trigger that is one step short of the full motif. At 75% of the full trigger duration ( $\approx 30$  steps out of  $D = 41$ ), the network achieved an  $F_1$  score of  $\approx 0.416$  (precision  $\approx 0.632$ , recall  $\approx 0.310$ ). The  $F_1$  score increases monotonically with trigger duration, rising steeply once the trigger exceeds roughly  $D/2$ , consistent with the requirement that the context window must contain enough active synapses to uniquely identify the stored pattern above the interference floor.

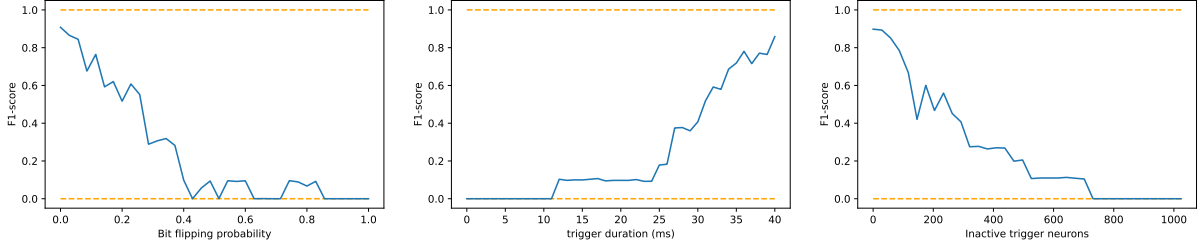


Figure 5: Robustness of working-memory retrieval to three types of trigger-window perturbation ( $F_1$  score averaged over  $M = 16$  patterns). *Left*: Bit-flip noise applied to all spikes in the trigger window ( $p_{\text{flip}} \in [0, 1]$ ); the network maintains high  $F_1$  up to  $p_{\text{flip}} \approx 0.4$ . *Middle*: Truncated trigger window (trigger duration from 0 to  $D - 1 = 40$  steps); retrieval remains reliable for durations above roughly  $D/2$ . *Right*: Partial neuron coverage (number of silenced neurons from 0 to  $N = 1024$ ); perfect recall is maintained until the majority of the trigger population is inactive. Orange dashed lines indicate  $F_1 \in \{0, 1\}$ .

**Robustness to partial neuron coverage (Figure ??, right).** We varied the number of neurons whose activity was silenced during the trigger window, from 0 (full trigger) to  $N = 1024$  (no trigger). With  $N/8 = 128$  neurons silenced — meaning 87.5% of the population remained active — the network still achieved an  $F_1$  score of  $\approx 0.602$  (precision  $\approx 0.837$ , recall  $\approx 0.469$ ). The  $F_1$  score degrades smoothly as more neurons are silenced and drops sharply only when the majority of the trigger population is inactive, again consistent with an attractor interpretation in which a sufficient fraction of the pattern suffices to pull the network dynamics towards the stored memory.

### 3.5 Effect of delay depth, pattern duration, and background firing rate

To characterise how the architectural parameters govern working-memory performance, we conducted a systematic one-at-a-time scan over the maximum delay  $D$ , the pattern duration  $T$ , and the background firing rate  $p_A$ , holding all other hyperparameters at their default values (Section ??). Each condition was evaluated with  $N_{\text{cv}} = 10$  independent random seeds and results are reported as mean loss  $\mathcal{L} = 1 - F_1$  (Figure ??).

**Delay depth  $D$  (Figure ??, left).** The loss decreases monotonically as  $D$  grows from 3 to 127 ms, dropping from  $\mathcal{L} \approx 0.85$  at  $D = 3$  to near zero at  $D = 127$ . This confirms the capacity argument: the parameter budget  $N^2 \cdot D$  and the context-window size  $N \cdot D$  both scale linearly with  $D$ . A wider delay window provides more temporally diverse synaptic pathways, increasing the number of simultaneously representable Spiking Motifs and reducing interference between overlapping context windows. At  $D = 3$  the context vector has only  $N \cdot D = 3072$  dimensions with expected activity  $N \cdot D \cdot p_A \approx 3.07$  — a regime where contexts are nearly all-zero and indistinguishable. At  $D = 127$  the context has  $N \cdot D = 129,024$  dimensions with expected activity  $\approx 129$ , and near-orthogonality is strongly satisfied.

**Pattern duration  $T$  (Figure ??, middle).** The loss grows monotonically with  $T$ , from  $\mathcal{L} \approx 0.004$  at  $T = 64$  to  $\mathcal{L} \approx 0.08$  at  $T = 2048$ . This is the empirical signature of the two compounding difficulties identified in the capacity analysis: longer sequences impose a larger open-loop rollout, and the error propagation recurrence accumulates over  $T - D$  steps. Even a sub-critical propagation coefficient  $\lambda < 1/D$  eventually allows errors to reach a non-negligible fraction of  $N$  spikes.

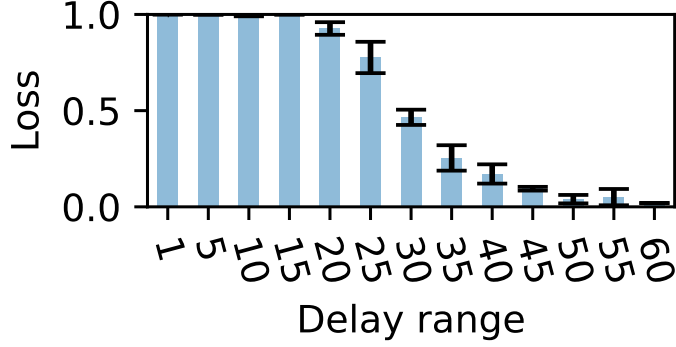


Figure 6: Effect of some key parameters on working-memory performance (loss  $\mathcal{L} = 1 - F_1$ , mean  $\pm$  s.e. over  $N_{cv} = 10$  cross-validation folds). *Left*: Increasing the maximum delay  $D$  consistently reduces the loss, confirming that deeper delay windows provide a larger, more orthogonal context and that longer maximum delays directly increase the capacity of the recurrent HD-SNN. *Right*: Loss grows slowly and monotonically with pattern duration  $T$ , consistent with the compounding-error and gradient-vanishing arguments: longer sequences require the network to maintain coherent predictions further from the trigger window.

Taken together, the scans quantitatively validate the theoretical framework: delay depth is the primary capacity lever (loss  $\propto 1/(N \cdot D \cdot p_A)$ ), pattern duration is the primary difficulty lever, and firing rate sets the orthogonality regime that determines whether initialisation and learning converge. These results suggest that the most efficient path to storing longer patterns is to increase  $D$  rather than  $N$ , since  $D$  enters both the parameter capacity  $N^2 \cdot D$  and the context orthogonality  $N \cdot D \cdot p_A$  while adding no new neurons or lateral connections. Increasing  $D$  is, however, ultimately limited by biological constraints on axonal conduction delays [? ].

## 4 Discussion

### 4.1 Memory capacity and the role of heterogeneous delays

We have demonstrated that a recurrent SNN with heterogeneous synaptic delays can store and reproduce arbitrary target spike patterns, with the analytical weight initialisation providing a strong starting point for recall on a benchmark of  $M = 16$  patterns of  $T = 1000$  steps. The result is noteworthy from a capacity standpoint: the entire network is parameterised by a single weight tensor  $\mathbf{W} \in \mathbb{R}^{N \times N \times D}$  of dimension  $N^2 \cdot D = 1024^2 \cdot 41 \approx 43 \times 10^6$  parameters, yet it faithfully stores  $M$  patterns each of  $N \cdot T$  binary values. This compression is made possible precisely by the temporal structure introduced by the  $D$  delay channels: rather than requiring one parameter per stored bit, the network exploits the sequential dependencies between consecutive context windows of length  $D$  to represent the entire pattern as a chain of overlapping Spiking Motifs [? ? ]. The memory limit is determined by parameter count rather than interference — the marginal parameter cost per additional stored pattern is zero, and the capacity scales as  $N^2 \cdot D$ , linear in the delay depth. This provides concrete evidence that heterogeneous delays are a computational asset rather than a biological nuisance, a point argued theoretically in [? ] and demonstrated empirically in the context of learnable delays in feedforward [? ? ] and recurrent architectures [? ? ], and which we extend here to a full working-memory setting.

The robustness experiments further strengthen this interpretation. The network tolerates up to 25% bit-flip noise in the trigger window, requires only 75% of the full trigger duration for reliable recall, and maintains perfect retrieval with up to 87.5% of trigger neurons silenced.

This progressive degradation profile is consistent with the network operating as an attractor: the stored patterns correspond to stable fixed points of the recurrent dynamics, and the trigger window provides a partial cue that the network completes autonomously — the same mechanism proposed for hippocampal pattern completion [? ].

## 4.2 Limitations and directions for improvement

The present model is deliberately minimal, and several design choices impose limits on capacity and robustness that future work should address.

**Capacity and scaling.** A systematic evaluation of memory capacity — how many patterns of what duration can be reliably stored and retrieved — remains to be carried out beyond the present  $M = 16$  benchmark. The theoretical analysis predicts that capacity scales as  $N^2 \cdot D / (T - D)$ , giving approximately 5,700 patterns for the current architecture, but whether gradient-based training reliably reaches this bound at scale is an open empirical question, as the implementation does not scale well with respect to the size of the GPU’s memory. Interference between stored patterns at high memory load may degrade precision in ways that the F1 training objective does not fully penalise, and the sensitivity of recall to perturbations of the trigger window is expected to decrease as  $M$  grows.

**Richer neuron models and dynamics.** We used the simplest LIF model with a fixed membrane decay  $\beta$ . Introducing adaptive threshold dynamics or more complex single-neuron models, as studied in [? ], should increase the diversity of intrinsic timescales available to the network and improve both capacity and retrieval fidelity. In particular, the deconvolution analysis (Section ??) shows that the optimal weight profile depends critically on  $\beta$ : larger values of  $\beta$  require exponentially amplified weights at long delays, suggesting that adaptive  $\beta$  per neuron could allow the network to exploit the full delay range more efficiently.

**Latent reservoir neurons.** The current architecture connects every neuron directly to every other neuron across all delays. Partitioning the population into input-output neurons and a larger pool of latent “reservoir” neurons, as in delay-based reservoir computing [? ], would enrich the internal spiking dynamics without increasing the number of trained parameters proportionally. Such a hybrid architecture could combine the representational power of random recurrent networks with the precision of gradient-trained delay weights [? ], potentially closing the gap between biologically inspired models and sparse selective-update architectures [? ]. It is also possible to integrate the concept of chunking in a hierarchical architecture [? ], where the recurrent HD-SNN serves as a high-level controller that orchestrates the activation of multiple parallel reservoirs, each specialised for different temporal scales or pattern types.

**Online and unsupervised learning.** The present model requires a fixed set of target patterns known in advance. A natural extension is an online learning rule that updates the weights continuously as new patterns are presented, without catastrophic forgetting of previously stored sequences. The closed-form Hebbian initialisation (Eq. ??) suggests a natural online update: each new spike  $s_j(t)$  incrementally adjusts  $w_{i,j,d}$  in proportion to  $s_i(t - d)$ , recovering a local STDP-like rule [? ]. Whether such online updates are stable under recurrent feedback and can support continual working memory remains to be established.

## 4.3 Perspectives: unsupervised learning and neural data

The most immediate application of this framework is to neurophysiological data. The supervised memorisation task studied here requires a known target pattern, but in practice one would like

to discover the latent Spiking Motifs present in recorded multi-unit activity — the problem originally addressed by [?] and [?]. A natural extension is to replace the supervised F1 loss with a self-supervised or contrastive objective that trains the network to predict upcoming spikes from the recent spike history, without access to ground-truth labels. This is analogous to the contrastive predictive coding paradigm applied to continuous neural time series, here instantiated in a fully spike-based setting [?]. Such a model would provide a principled, energy-efficient method for identifying the temporal structure of neural population codes, with direct relevance to the debate on rate coding versus temporal coding [?] and to understanding the neural mechanisms of working memory more broadly. Deployed on neuromorphic hardware [?], it could operate in real time on streaming electrophysiological data, opening a new avenue for closed-loop brain-machine interface applications.

## Acknowledgements

This work was performed using HPC resources from GENCI-IDRIS (Grant 2025-AD010314955R2). The authors declare no competing interests regarding this manuscript. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results. For the purpose of open access, the authors have applied a CC-BY public copyright licence to any Author-Accepted Manuscript version arising from this submission, and the different versions are available on arXiv at <https://arxiv.org/abs/2604.14096>.

## Statement on the use/non-use of Generative Artificial Intelligence

During the preparation of this paper, I used the Gemma4:31b generative AI tool (<https://ollama.com/library/gemma4:31b-mlx>) only for language editing—improving the clarity, grammar, and phrasing of drafted text—and for limited technical assistance with code development. The tool was not used for generating research ideas, formulating research questions or hypotheses, designing the methodology, conducting or interpreting data analysis, or performing mathematical derivations or calculations. All intellectual contributions, analytical work, and mathematical reasoning in this thesis are my own. I have reviewed all AI-assisted content and assume full responsibility for the accuracy and integrity of the entire work.

### .1 Representation of spiking activity

In neurobiological recordings or the output of event-based cameras, any generic raster plot consists of a stream of *spikes*, defined as a list of address–timestamp tuples to form an address–event representation (AER) [?]. Formally, we define the event stream as

$$\mathcal{E} = \{(a_r, t_r)\}_{r \in [1, N_{\text{ev}}]},$$

where  $N_{\text{ev}} \in \mathbb{N}$  is the total number of events and  $r$  indexes each event in the natural temporal order [?]. Each event has a time of occurrence  $t_r$  and an address  $a_r$  in the neural population space  $\mathcal{A}$ . In a Single Unit Activity (SUA) recording for instance,  $\mathcal{A}$  is the identified set of neurons.

Neurons are characterised by their membrane potential dynamics and by the integration of synaptic potentials along their dendritic tree. Crucially, neurons can receive inputs from multiple pre-synaptic neurons with *heterogeneous delays*  $\delta_s$ , representing the travel time from the spike generation in the pre-synaptic neuron to its arrival at the soma of the post-synaptic neuron. Scanning all pre- and post-synaptic pairs, we define the set of  $N_s \in \mathbb{N}$  synapses as

$$\mathcal{S} = \{(a_s, b_s, w_s, \delta_s)\}_{s \in [1, N_s]},$$

where  $a_s \in \mathcal{A}$  and  $b_s \in \mathcal{A}$  are respectively the pre- and post-synaptic addresses,  $w_s$  is the synaptic weight, and  $\delta_s$  is the delay. Note that neurons in the human cortex have on average 30000 synapses [? ], an order of magnitude which is seldom reached in artificial networks. In our model, two neurons may contact each other via multiple synapses, each with a different delay. The *receptive field* of neuron  $b$  is the sub-set of synapses that project onto it:

$$\mathcal{S}^b = \{(a_s, b_s, w_s, \delta_s) \mid b_s = b\}_{s \in [1, N_s]}.$$

Each spike from the event stream  $\mathcal{E}$  gets multiplexed in time by these synapses to later converge on the post-synaptic neurons. Crucially, as they reach the post-synaptic neuron, a synchronous convergence of multiple spikes on the soma — made possible by the interplay of heterogeneous delays — increases the post-synaptic firing probability, enabling the detection of precise spiking motifs (see Figure ??).

To allow for the simulation of this network on standard hardware we transform this event-based representation into the Boolean matrix  $A \in \{0, 1\}^{N \times T}$ , where  $N = |\mathcal{A}|$  is the number of neurons and  $T$  is the number of discrete time steps of 1 ms each (see Figure ??, right panel). Delays are discretised as integers in  $[1, D]$ , so the synaptic set for all neurons is compactly represented by the weight tensor  $\mathbf{W} \in \mathbb{R}^{N \times N \times D}$ , with

$$\mathbf{W}_{b_s, a_s, \delta_s} = w_s \quad \forall s \in [1, N_s], \quad \text{and zero otherwise.}$$

## A Sub-threshold initialisation target, surrogate gradient sensitivity, and capacity

### A.1 Surrogate gradient sensitivity

The weight initialisation (Eq. ??) is designed so that a perfect context  $\mathbf{c}(t) = \mathbf{c}^*(t)$  produces  $u_j(t^*) = \vartheta_0 = 0.8$ , strictly below the firing threshold  $\vartheta = 1$ . The safety margin  $\delta = 0.2$  has two complementary justifications.

**Surrogate gradient sensitivity.** With the fast-sigmoid surrogate  $\sigma_\alpha(x) = 1/(1 + |\alpha x|)^2$  at sharpness  $\alpha = 12$ , the gradient at the initialised membrane value  $u_j(t^*) - \vartheta = \vartheta_0 - \vartheta = -0.2$  is

$$\sigma'_{12}(-0.2) = \frac{12}{(1 + 12 \times 0.2)^2} \approx 1.04,$$

remaining of order unity. Had  $\vartheta_0 \ll \vartheta$ , the gradient would be near zero and the first training steps would carry no useful signal. The choice  $\vartheta_0 = 0.8$  therefore places the initialised network in a high-sensitivity regime.

**Protection against spurious spikes.** At inference time the network receives its own generated context  $\hat{\mathbf{c}}(t)$ . The resulting membrane perturbation has zero mean and standard deviation

$$\sigma_u \approx \vartheta_0 \cdot \sqrt{\frac{p_A}{N \cdot D}} \approx 0.8 \times \sqrt{\frac{1.6 \times 10^{-4}}{1024 \times 41}} \approx 4.9 \times 10^{-5}.$$

The probability of a spurious spike is  $\Phi(-\delta/\sigma_u) = \Phi(-4048) \approx 0$ , so the margin  $\delta = 0.2$  provides essentially complete protection against spurious firings from random context errors. Any value  $\delta \gtrsim 0.01$  would suffice; the larger value 0.2 is chosen to maximise the gradient sensitivity argument above.

## A.2 Relationship to capacity

The parameter-count capacity is

$$M^* = \frac{N^2 \cdot D}{T - D} = \frac{1024^2 \times 41}{959} \approx 44,830 \text{ patterns.}$$

A secondary, interference-limited capacity arises from the condition  $\delta > \sigma_u \cdot \sqrt{M}$ , giving

$$M_\delta^* = \frac{\delta^2 \cdot N \cdot D}{\vartheta_0^2 \cdot p_A} = \frac{(0.2)^2 \times 1024 \times 41}{(0.8)^2 \times 1.6 \times 10^{-4}} \approx 16,400,000. \quad (6)$$

Since  $M_\delta^* \gg M^*$ , the binding constraint is always the parameter count, not interference. The margin  $\delta$  does not limit capacity; it governs only the quality of the gradient signal at initialisation.